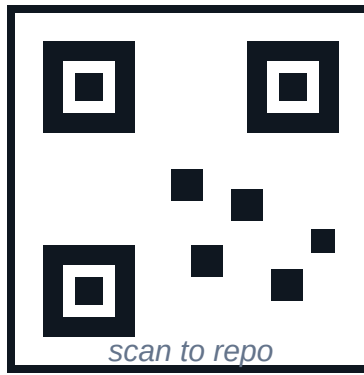


# Statistical Screening for Poisoned Machine-Learning Training Data



## Detection and Quarantine of Suspicious Samples at the Data-Ingestion Boundary

Pooja Kiran

Security Engineering • AI / Agent Security • Cloud Security

MIT • Python 3.12 • HEAD b45d662 • verified 2026-09-26

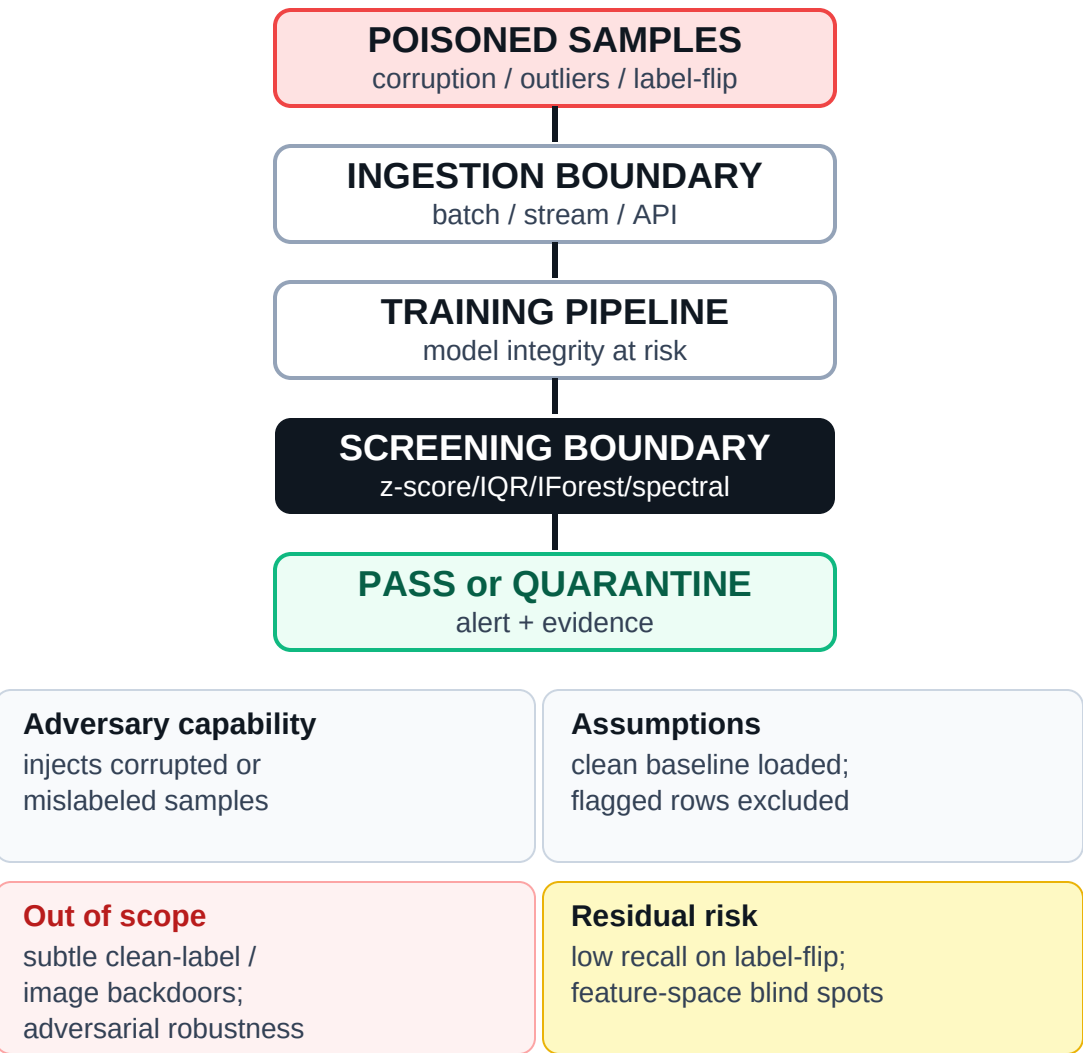
### RESEARCH QUESTION

How effectively can statistical screening identify poisoned or corrupted training samples before they enter the training pipeline?

### 01 • THE FAILURE MODE

Poisoned or corrupted samples enter training silently. A vendor feed drifts or a shared table gets bad rows, and precision quietly drops over retraining cycles — the pipeline never errors, so nobody checks the data. A screening layer at ingestion (MITRE ATLAS AML.T0020) can catch gross corruption — but not subtle attacks.

### 02 • THREAT MODEL



### 03 • OBJECTIVE

Determine how well an unsupervised statistical ensemble screens ingestion data — and measure explicitly where it fails.

- 01 – Z-score + IQR + IsolationForest
- 02 – Spectral signatures (label-aware)
- 03 – Majority-vote ensemble + quarantine
- 04 – Streaming (Welford) + drift

### HERO • INGESTION SCREENING ENSEMBLE

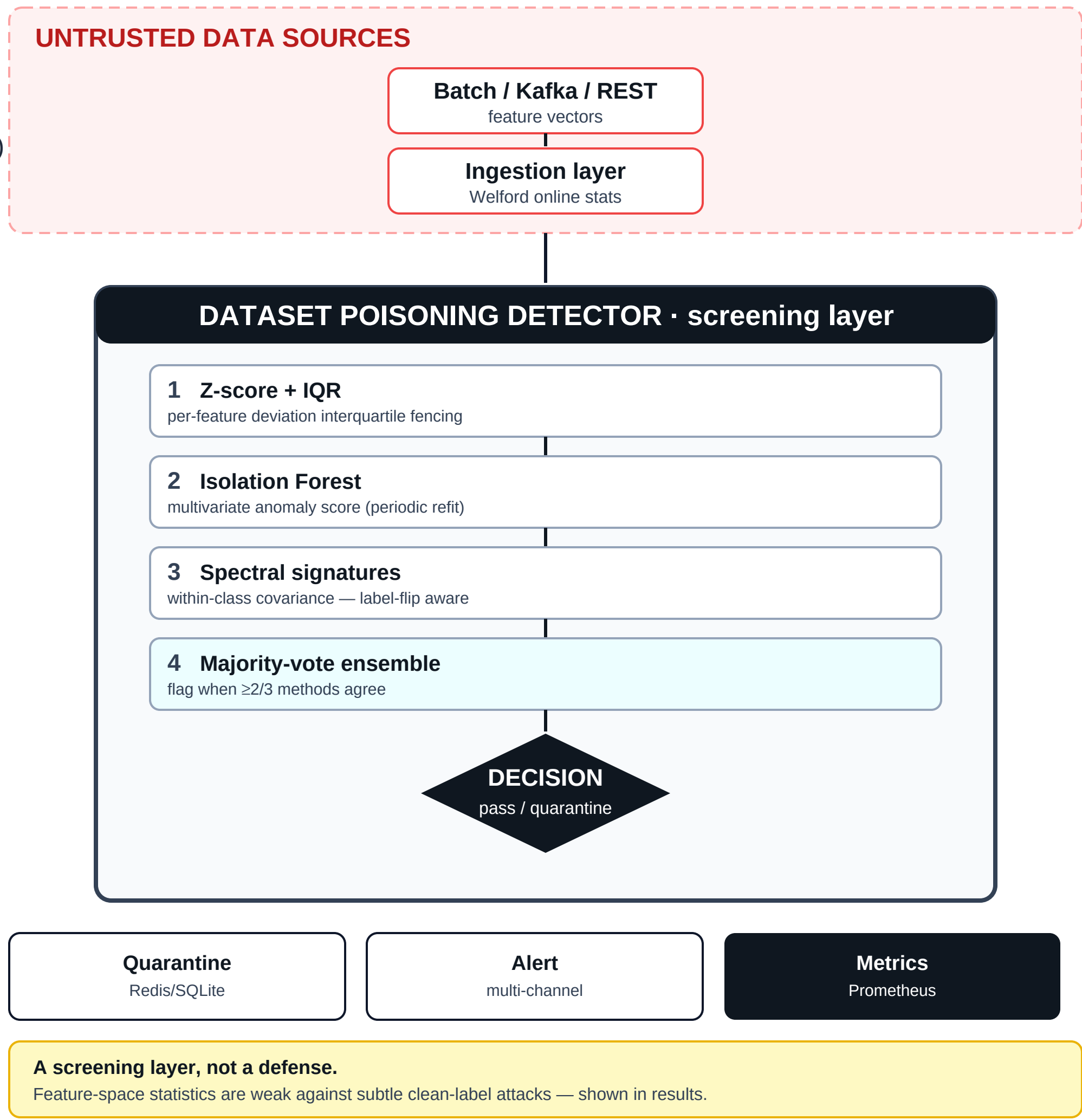
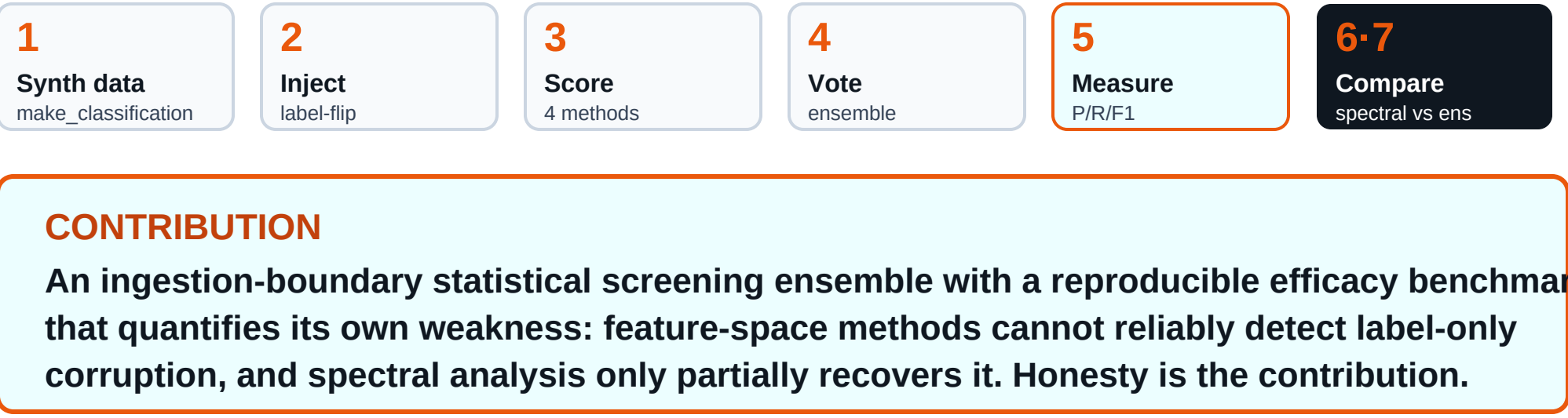


Figure 1. Ensemble screening at the ingestion boundary; spectral analysis targets label-flip attacks feature-space methods miss.

### METHOD • EFFICACY BENCHMARK



### RESULTS • WHERE SCREENING FAILS (measured)



Label-flip benchmark, seed 2018, 2000×100 synthetic. F1 rises with contamination but stays low — by design honest.

| Spectral F1 vs contamination |      |
|------------------------------|------|
| @ 5% poison                  | 0.08 |
| @ 10% poison                 | 0.23 |
| @ 20% poison                 | 0.37 |

### SPECTRAL vs ENSEMBLE F1 (@20%)



Spectral beats ensemble on 2/3 rates; both remain weak on label-flip. spectral\_benchmark.json

### BENCHMARK PROVENANCE

results/spectral\_benchmark.json (committed). Tran et al. 2018 setup. Throughput = environment-scoped microbenchmark; no production SLA claimed.

### NOT ESTABLISHED BY THIS REPO

High recall on label-flip attacks. Detection of subtle clean-label / image backdoors. Robustness guarantee.

### SECURITY BOUNDARY

| PROTECTS / DETECTS                                                                                                                                                           | DOES NOT ESTABLISH                                                                                                                                     |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>gross outliers</li><li>obvious corruption</li><li>distribution shift</li><li>replay (fingerprint)</li><li>provenance logging</li></ul> | <ul style="list-style-type: none"><li>subtle clean-label attacks</li><li>image backdoors</li><li>targeted poisoning</li><li>robustness proof</li></ul> |

**SCREENING ≠ DEFENSE**

### LIMITATIONS

- 1. Low recall on label-flip (F1 0.08–0.37 measured).
- 2. Feature-space methods blind to clean-label attacks.
- 3. Streaming stats are per-feature; correlations lost.
- 4. Synthetic benchmark ≠ real pipeline data.
- 5. First-pass filter, not adversarial robustness.

### FUTURE WORK

- Evaluate on representative real pipeline data.
- Label-aware detectors beyond spectral.
- Multivariate streaming statistics.
- Clean-label attack detection research.
- Calibrated thresholds per dataset.

### CONCLUSION

The tool screens ingestion data with a 4-method ensemble and quarantines outliers. Its committed benchmark deliberately shows weak label-flip F1 (0.08–0.37), documenting exactly where statistical screening breaks down. It is a filter, not a defense.

**REPRODUCE**  
pytest tests/  
python benchmark/cifar10\_label\_flip\_benchmark.py